Obligatory Assignment 2

Lars Haukland

March 20, 2025

Abstract

Creating two parallel bfs and testing speedup. In the following obligatory have I created two parallel breadth-first search algorithms. One simple, which only parallelizes the exploration of each level. And another, which first runs a few iterations of sequential bfs and then switches to parallel and tries to load balance the threads equally.

1 Algorithm outline

This section outlines the algorithms and tries to explain the idea behind them.

1.1 Simple parallel bfs

This simple parallel bfs explores layer by layer and for each layer it distributes the vertices among the threads. The threads then discover the neighbors of those threads, and then all discovered neighbors are put into the next layer. A problem with this approach is that the degree of vertices can be very varying, so if the graph is very dense then this simple approach can work nicely. But most realistic graphs are not very dense, so the number of neighbors each thread will visit will vary alot making it not very balanced. The runtime is not easy to really get exact because of the graphs structure really determines the runtime, but see the experiments section for speedup.

1.2 Alternative parallel bfs

Here the bfs is not that simple anymore. An idea agains the simple parallel bfs is that it is expensive to distribute the vertices every single layer. What if we could just give the threads some vertices and then let them explore their own part of the graph. This is nice but stumbles on to a very problematic situation because most graphs are not nicely balanced. This way some threads will have a larger part of the graph. This algorithm tries to fix this while still letting threads run their bfs.

The general idea is to first run sequential bfs n times to get some initial vertices to search. I choose $n = 6$, there's really no good reason behind this but I didn't see too much of an effect on the runtime from this. Then each thread gets given some vertices to do bfs from. The threads then do that, putting the next layer into another array and copying that back to search those vertices next iteration.

Then comes the load balancing. If at any point, after exploring the next layer, a thread has way more vertices to search than the other threads. All vertices to search will then be gathered and then re distributed to the vertices in an evenly manner. For determining when the load is imbalanced I choose to go with the criteria that if the max load of a vertice is 50% larger than the average load then they will be rebalanced. This can also be written as
$let\ k = 1.5;$
$let\ rebalance\_condition = max\_load \geq k * avg\_load$

It is obvious that the smaller the k, the more aggressive the balancing. I think I choose to small of a k, but it's hard too test the performance. As this should be done with the large files, but this takes a while and its a lot of output to analyse.

2 Experiments

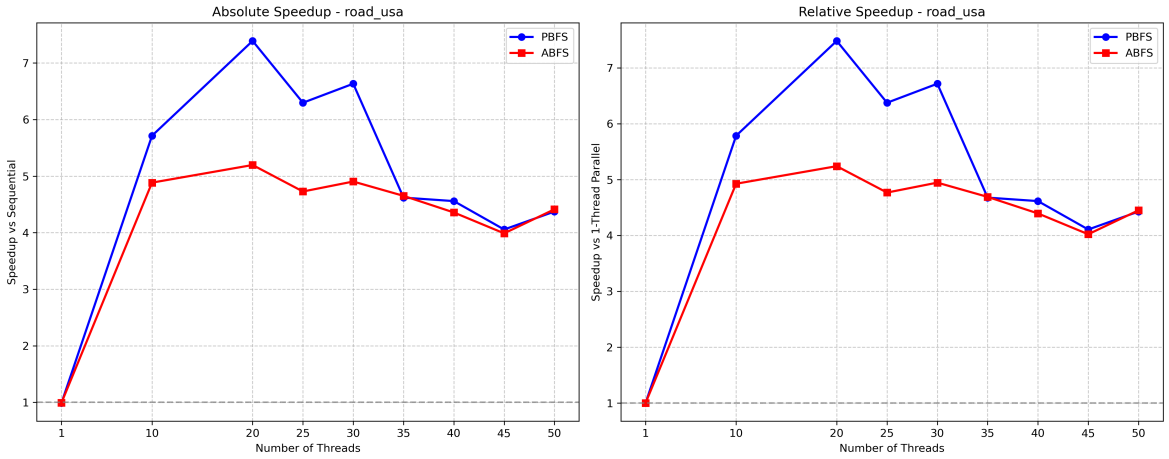
For the experiments I have ran *driver.c* with the *data_files* file. The specified configuration was to run 4 files (given in task), each 3 times (choosing the best result) and with 9 different thread configurations.

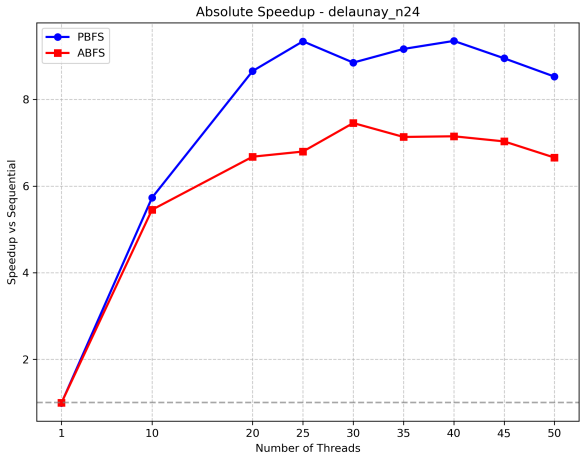
I edited the *driver.c* file to make it print out latex tables with the run time data. This is in the function *store_p_value_latex* but PLEASE NOTE: This function was not made by me. I don't have enough c-competance, latex-competance, or the time. This function was generated using the Claude AI model. I also plotted the data in python. This python script was also made by the Claude AI model. I did this to save time and focus on writing the report instead of trying to make latex tables or plots.

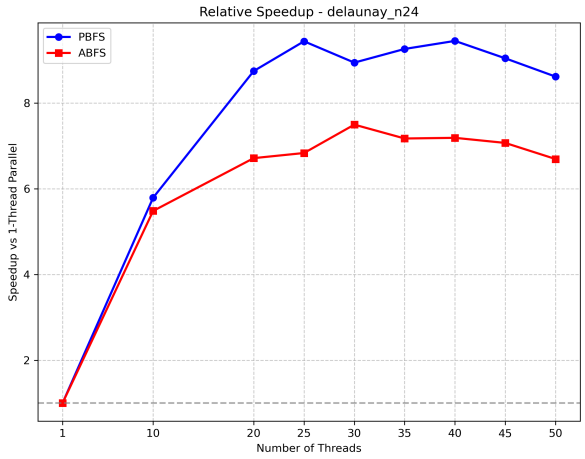
Now below are plots of the speedup compared to the sequential bfs and the relative speedup compared to one thread. This is for each graph file.

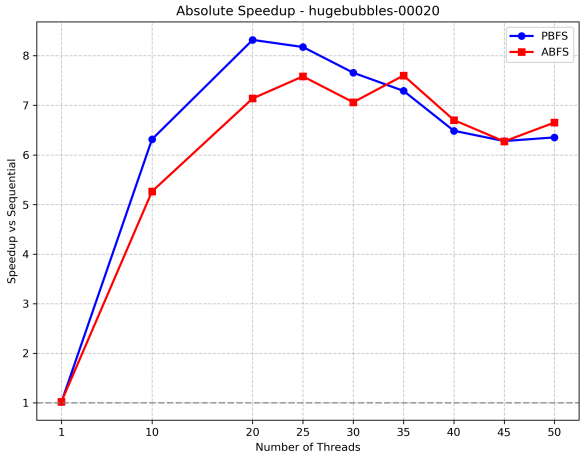
From the results we can see that in fact the alternative parallel bfs was worse for most of the runs and files. There are some conclusions we can draw from the data:

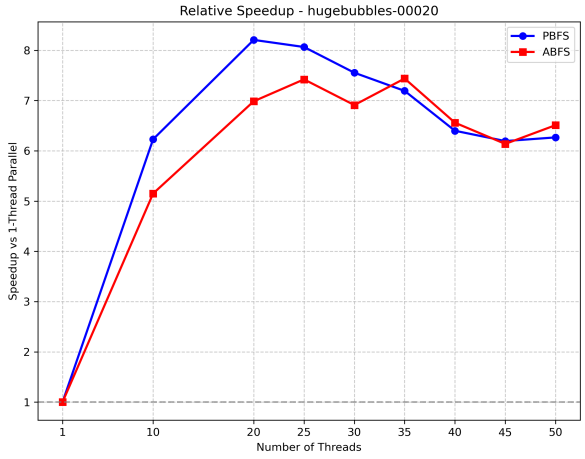
1. ABFS is better when the number of threads is high. Compared to PBFS, ABFS has never better speedup except for sometimes when $p > 25$.
2. On graphs with $|V|$ low compared to $|E|$, PBFS performs better than ABFS. This may be a little far fetched conclusion but looking at the plots the dataset *delaunay_n24* har a large gap between PBFS and ABFS speedup. This is also the graph with $|V| = 16777216, |E| = 50331601$. Compared to the other graphs this graph has a distinct gap between the number of vertices vs the number of edges.
3. The ideal p in terms of scaling for PBFS lies in the range of 20 ± 5. While for ABFS this varies per graph. This is interesting.

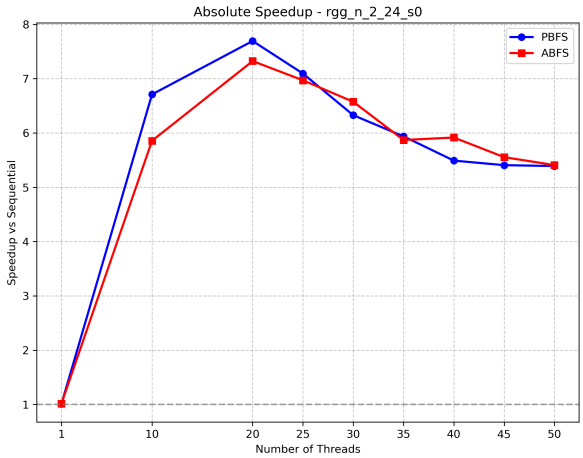


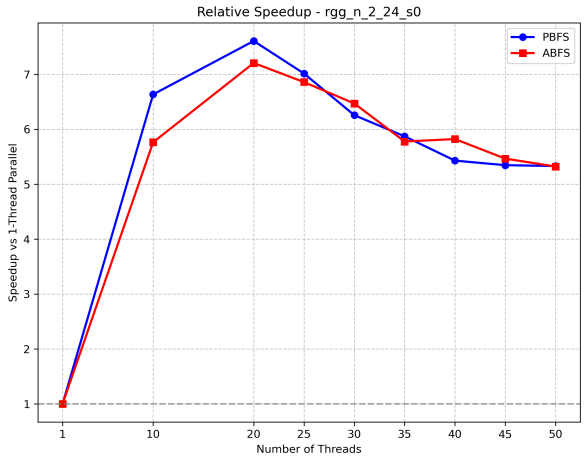

Absolute Speedup - delaunay_n24
PBFS
ABFS
Speedup vs Sequential
Number of Threads


Relative Speedup - delaunay_n24
PBFS
ABFS
Speedup vs 1-Thread Parallel
Number of Threads


Absolute Speedup - hugebubbles-00020
PBFS
ABFS
Speedup vs Sequential
Number of Threads


Relative Speedup - hugebubbles-00020
PBFS
ABFS
Speedup vs 1-Thread Parallel
Number of Threads


Absolute Speedup - rgg_n_2_24_s0
PBFS
ABFS
Speedup vs Sequential
Number of Threads


Relative Speedup - rgg_n_2_24_s0
PBFS
ABFS
Speedup vs 1-Thread Parallel
Number of Threads

3 Appendix

Table 1: Parallel BFS - road_usa ($|V| = 23947347$, $|E| = 28854312$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.584881	1.00	—
1	1.604629	0.99	1.00
10	0.277390	5.71	5.78
20	0.214463	7.39	7.48
25	0.251700	6.30	6.38
30	0.238942	6.63	6.72
35	0.343094	4.62	4.68
40	0.347714	4.56	4.61
45	0.390765	4.06	4.11
50	0.362409	4.37	4.43

Table 2: Parallel BFS - delaunay_n24 ($|V| = 16777216$, $|E| = 50331601$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.368865	1.00	—
1	1.382842	0.99	1.00
10	0.238847	5.73	5.79
20	0.158142	8.66	8.74
25	0.146521	9.34	9.44
30	0.154668	8.85	8.94
35	0.149344	9.17	9.26
40	0.146378	9.35	9.45
45	0.152951	8.95	9.04
50	0.160529	8.53	8.61

Table 3: Parallel BFS - hugebubbles-00020 ($|V| = 21198119$, $|E| = 31790179$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	2.813991	1.00	—
1	2.776183	1.01	1.00
10	0.445595	6.32	6.23
20	0.338240	8.32	8.21
25	0.344169	8.18	8.07
30	0.367489	7.66	7.55
35	0.385952	7.29	7.19
40	0.433809	6.49	6.40
45	0.448165	6.28	6.19
50	0.442961	6.35	6.27

Table 4: Parallel BFS - rgg_n_2_22_s0 ($|V| = 4194304$, $|E| = 30359198$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.049931	1.00	—
1	1.038403	1.01	1.00
10	0.156478	6.71	6.64
20	0.136477	7.69	7.61
25	0.148004	7.09	7.02
30	0.165943	6.33	6.26
35	0.176832	5.94	5.87
40	0.191224	5.49	5.43
45	0.194180	5.41	5.35
50	0.194764	5.39	5.33

Table 5: Alternative Parallel BFS - road_usa ($|V| = 23947347$, $|E| = 28854312$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.584881	1.00	—
1	1.598266	0.99	1.00
10	0.324489	4.88	4.93
20	0.305016	5.20	5.24
25	0.335243	4.73	4.77
30	0.323191	4.90	4.95
35	0.340788	4.65	4.69
40	0.363755	4.36	4.39
45	0.397452	3.99	4.02
50	0.359143	4.41	4.45

Table 6: Alternative Parallel BFS - delaunay_n24 ($|V| = 16777216$, $|E| = 50331601$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.368865	1.00	—
1	1.375951	0.99	1.00
10	0.251043	5.45	5.48
20	0.204979	6.68	6.71
25	0.201393	6.80	6.83
30	0.183606	7.46	7.49
35	0.191885	7.13	7.17
40	0.191469	7.15	7.19
45	0.194692	7.03	7.07
50	0.205614	6.66	6.69

Table 7: Alternative Parallel BFS - hugebubbles-00020 ($|V| = 21198119$, $|E| = 31790179$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	2.813991	1.00	—
1	2.754033	1.02	1.00
10	0.534713	5.26	5.15
20	0.394210	7.14	6.99
25	0.371057	7.58	7.42
30	0.398509	7.06	6.91
35	0.370210	7.60	7.44
40	0.419829	6.70	6.56
45	0.448694	6.27	6.14
50	0.423002	6.65	6.51

Table 8: Alternative Parallel BFS - rgg_n_2_22_s0 ($|V| = 4194304$, $|E| = 30359198$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.049931	1.00	—
1	1.033434	1.02	1.00
10	0.179274	5.86	5.76
20	0.143367	7.32	7.21
25	0.150642	6.97	6.86
30	0.159709	6.57	6.47
35	0.178846	5.87	5.78
40	0.177480	5.92	5.82
45	0.189029	5.55	5.47
50	0.194130	5.41	5.32